

On-Chip Spike Detection and Classification using Neural Networks and Approximate Computing

Efstratios Zacharelos
DIETI

University of Napoli Federico II
Napoli, Italy
efstratios.zacharelos@unina.it

Ciro Scognamiglio
DIETI

University of Napoli Federico II
Napoli, Italy
ciro.scognamiglio@unina.it

Ettore Napoli
DIEM

University of Salerno
Salerno, Italy
enapoli@unisa.it

Diego Gragnaniello
DIEM

University of Salerno
Salerno, Italy
digragnaniello@unisa.it

Abstract—Neural ensembles control sensory, motor, and cognitive functions. Action potentials of neuronal cells (spikes) may signify such functions, or the presence of a pathology. In this paper we give the circuitual implementation of an Artificial Neural Network, able to sort (detect and classify) spikes in real time. The system is synthesized targeting a 14nm FinFET technology. To partially alleviate the computational burden, approximate computing methods have been integrated during the inference stage, yielding up to 63% reduction in dynamic power. The different versions of the circuit reach an accuracy range from 65% to 93%, with silicon area and power that range from $2000\mu\text{m}^2$, $0.1\mu\text{W}@30\text{kHz}$ to $6000\mu\text{m}^2$, $0.7\mu\text{W}@30\text{kHz}$. The electrical performances of the proposed circuit overcome the state of the art of spike detection circuits while providing the additional feature of spike sorting in a single integrated solution.

Keywords—Neural spike detection, Neural spike sorting, Neural spike classification, VLSI, Machine learning, Artificial neural networks, Approximate computing

I. INTRODUCTION

Significant advancements have been made in Brain Machine Interfaces (BMIs) since the initial proposal of devices directly linked to ensembles of cortical neurons [1], [2]. There are various ways to detect neural activity [3], including electroencephalography, functional magnetic resonance imaging, electrocorticography (ECoG), and others [4]. Implantable approaches to electrically contact the cortical tissue, like ECoG and penetrating microelectrode arrays (MEAs), provide better results with respect to non-invasive techniques, due to a significantly higher signal-to-noise ratio. Also, even though it is possible to observe neural activity exploiting different means of recording, like optical, and magnetic [5], electrical recording is the most commonly used.

The electrical charge of the brain is maintained by billions of electrically charged neurons. With the help of transport proteins, neurons pump ions across their membranes. This effect can lead to a wave, known as volume conduction. The neural information is then received by MEAs and recorded on chip. Progress in integrated circuits and microsystem technologies has made implanting thousands of intracortical electrodes possible, allowing researchers to investigate bigger neural ensembles. Nevertheless, transmitting massive data wirelessly for external post-processing poses unrealistic bandwidth and power requirements [6]–[8].

Spike detection algorithms mitigate the problem by not streaming the whole raw signal. Instead, it is processed on-site, spikes are detected and possibly classified in different categories, and only that information is transmitted off-brain [6]. After that, verification and statistical post-processing can be performed without heavy electrical constraints [9].

The detection and classification of neural spikes constitute a spike sorting enabled system, capable of attributing individual spikes to individual neurons. This field has gained significant interest in neuroscience due to the advances in implantable microelectrode arrays, that combined with efficient and accurate spike sorting systems, are essential for BMIs, experimental neural prosthetics, real-time neurological disorder monitoring, and neuroscience research [10].

An analog front-end for spike sorting systems is proposed in [11]. A Low Noise Amplifier is employed to amplify the signals of the electrodes, from the sub-mV to the mV range. A bandpass filter is then used to: a) reject the local field potential (LFP) (high pass) and b) avoid aliasing (low pass). Finally, an ADC provides the system with a digital signal. Once the spikes exceed the LFP and background noise, they can be detected using amplitude thresholding [7], [8], [11]–[15]. In [7] and [8], an adaptive threshold is utilized, after two cascaded energy operators for spike enhancement, specifically the Absolute Differential Operator and the Amplitude Slope Operator. A novel software-based spike extraction method, called spike detection by differences, is proposed in [16].

Different proximity and orientation with respect to detecting electrodes, result in different neural profiles. Feature extraction can be used to perform spike sorting. Template matching [11]–[14], [17] is a common technique which measures the similarity index between a spike and a template. The authors in [18] propose an iterative Bayesian approach to separate the LFP from the spiking activity, using prior information about the power spectral density of the LFP. Other methods for spike sorting are Principal Component Analysis, First and Second Derivative Features Extraction [11], K-means, and Superparamagnetic Clustering. Lately, machine learning approaches have also been considered for spike detection and classification [15], [19], [20].

Some of the systems in the literature provide real-time functionalities as they can detect and/or sort spikes at run time. Moreover, authors have implemented their contributions in ASIC technology, FPGAs, or developed software algorithms. Table I summarizes most of the aforementioned information.

In this paper, machine learning approaches are used to detect and classify as Type A, B or C the spiking activity in simulated recordings of brain activity. The three types of spikes correspond to different firing neurons in an investigated cube of brain tissue. The trained network is described in Verilog and synthesized in a 14nm technology. The innate error factor in machine learning and the vast number of multiplications allow the fruitful introduction of approximate multiplying circuits [21]–[23], to significantly reduce silicon area and power dissipation. Such approximate designs are frequently used in error tolerant applications, to improve the electrical performance by allowing imprecise results.

TABLE I: Literature Review. The proposed design offers real-time spike detection and classification with a NN hardware implementation.

	Spike Detection	Spike Classification	Real-Time	NN	Implementation	Key electrical performance
[7]	Absolute Differential Operator	NO classification	Yes	No	ASIC 28nm	2100 $\mu\text{m}^2/\text{ch}$ 0.15 $\mu\text{W}/\text{ch}$
[8]	Amplitude Slope Operator Adaptive thresholding					230 $\mu\text{m}^2/\text{ch}$ 1.4 $\mu\text{W}/\text{ch}$
[11]	Non-linear Energy Operator Absolute Value Thresholding Amplitude Threshold	Template Matching Principal Component Analysis 1st/2nd Derivative Features	Yes (front-end)	No	ASIC 0.18 μm (front-end)	0.16mm $^2/\text{ch}$ 10-200 $\mu\text{W}/\text{ch}$ (front-end)
[12]	Amplitude Threshold	Template Matching	Yes	No	Chip / FPGA	hundreds of mW
[13]	Amplitude Threshold	Template Matching	Yes	No	Chip / Software	~5mW
[14]	Amplitude Threshold	Template Matching	No	No	Software	-
[15]	Amplitude Threshold	NO classification	Yes	Yes	Software	-
[16]	LFP filtering and "SDD"	NO classification	Yes	No	Software	-
[17]	Detection and classification with Template Matching		Yes	No	Software	-
[18]	HF thresholding	Iterative Bayesian approach	No	No	Software	-
[19]	Detection and classification with Convolutional Neural Network		Yes	Yes	Software	-
[20]	Detection and classification with K-means		No	Yes	Software	-
Proposed	Detection and classification with Neural Network		Yes	Yes	ASIC 14nm FinFET	0.278 μW 3938 μm^2 >90% accur. 4.8 3 Tab. III

The proposed implementation shows that by using a small integrated neural network, it is possible to obtain good accuracy in both detection and classification of the neural spikes whilst using a circuit with limited requirements in terms of both silicon area and energy. When compared to the state of the art, the proposed circuit is the only integrated solution employing a neural network that provides both detection and classification. In terms of electrical performance, the proposed designs overcome the state of the art. As an example, one of the proposed circuits obtains more than 90% accuracy in detection and classification with a silicon area occupation of 3938 μm^2 and power dissipation of 0.278 μW .

The remainder of the paper is organized as follows. In section II the dataset and the developed models are introduced. The hardware implementation and the electrical performances are in section III. The conclusions are drawn in section IV.

II. ARTIFICIAL NEURAL NETWORK

A. Dataset

NeuroCube, presented in [24], is a freely available and commonly used [7], [9], [11], [13], [17], [18], method to generate realistic simulations of extra-cellular recordings. The simulations are obtained by superimposing the activity of neurons, that are randomly placed in a cube of brain tissue and recorded by an electrode. The recordings include noise and consist of labelled data that provide information about the presence and type of spikes. The simulated data in this paper exploit 30-second records of noisy data sampled at 5kHz and contain around 1000 spikes for each class. For this data the common technique of amplitude thresholding is limited to quite low detection accuracy (and does not allow sorting).

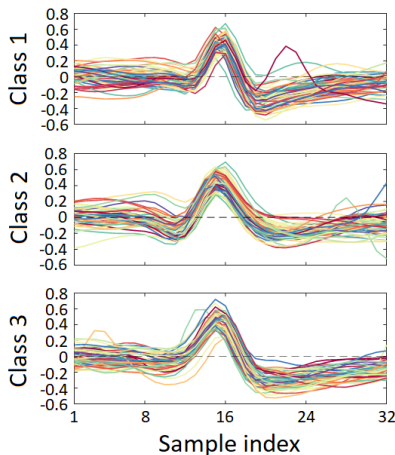


Fig. 1. Superimposed spikes for the three different classes.

The energy of the spikes is usually well concentrated in 32 continuous samples. Therefore, we aimed at the development of a NN with 32 input neurons that receive the incoming "window" (one sample per neuron). The NN has four output neurons (one per class and one for the no-spike) that should turn *high* to indicate the type or absence of spike, respectively. As mentioned, NeuroCube defines three different types of spikes, exemplified in Fig. 1 with a normalized amplitude.

The 32-sampled windows that contain a spike labelled by NeuroCube have been extracted from the recorded data. Data augmentation has been carried out by extracting each spike 13 times shifting the peak value of the spike from the 10th sample to the 22nd sample. In this way the total number of windows with a labelled spike has been multiplied by 13, reaching approximately 39000 windows containing a spike. In this way, the network is also trained to detect spikes even if they are not perfectly centered, thus developing a more robust behavior.

Since most samples in the generated dataset (i.e. 83%) belong to the no-spike class, a random undersampling is adopted to balance the dataset, extracting around 13000 windows that do not contain a spike. Finally, the used dataset is composed by 52000 windows equally divided into normal brain activity, spikes of Type-A, B, and C, respectively. The dataset developed for this study is made available online, [25].

B. Design of the Neural Network

TensorFlow has been used to train and validate the NNs exploiting the exhaustive cross-validation protocol. Specifically, the balanced dataset has been split into 10 folds. At each training stage, 9 folds have been used for training and the last one for validation. Finally, the average accuracy over the 10 folds has been extracted for performance comparison. By doing this, the whole dataset is used for both training and validation, thus avoiding a performance bias. Note that the adopted processing pipeline avoids putting different augmented versions of the same signal in both the training and the validation folds, which would falsely boost the results.

The training has been conducted over 150 epochs using the RMSprop optimized with a learning rate equal to 0.0015 and a batch size equal to 512. However, the training was designed to stop if the validation loss shows no improvement for 5 consecutive epochs. In every case, the models achieving the best validation loss have been retained.

The chosen activation function is the Rectified Linear Unit (ReLU) which proved efficient to model in hardware. As a loss function, the categorical cross-entropy has been used, which is ideal for multi-class classification models where

TABLE II: Number of parameters, and validation accuracies for different architectures.

Neurons in Layer				Number of Parameters	Validation Accuracy
In	1	2	Out		
32	16	-	4	596	88.57 %
32	20	-	4	744	88.34 %
32	24	-	4	892	88.01 %
32	24	8	4	1028	95.54 %
32	24	16	4	1260	96.17 %

used as an output layer, to convert a vector of 4 real numbers into a probability distribution of the 4 possible outcomes.

Gathering trainable parameters around a specific value can result in a more concise binary representation, that will in turn result in a smaller quantization error (introduced by the hardware implementation of the NN). Therefore, a regularization layer has been added during training in TensorFlow. L2 regularization adds the “squared magnitude” of the coefficient as a penalty term to the loss function, thus forcing parameters to be smaller but not exactly zero. As a result, the parameters have been arranged closely around 0. In this way, less bits are required for the representation of the integer part of the numbers, allowing us to spare more bits for the fractional part and increase the numerical accuracy.

Table II shows some of the most prominent architectures that have been developed. Understandably, accuracy grows significantly by adding an extra layer. The architecture chosen to be implemented in an ASIC, is the most accurate shown in Table II, featuring two hidden layers, with 24 and 16 neurons respectively. The validation accuracy is obtained on a set of data that is not used during learning, to effectively test the generalization ability of the model.

III. HARDWARE IMPLEMENTATION

A. Proposed Architecture

The proposed model has been described in Verilog in a fully parameterized way, enabling effortless modifications. In fact, with a few adjustments the code can be adapted to accommodate any dense NN. Each neuron is responsible for calculating all the corresponding weights - inputs products and adding them to the bias. In a realistic scenario, the 32-sampled window is processed one sample at a time. The rightmost sample of a waveform is read first and multiplied by the 24 first weights of each neuron in the first hidden layer. Each of the remaining 31 inputs that follow, are multiplied by 24 corresponding weights in the respective neuron. Once all the 32 inputs have entered the system, a new window follows. By processing one input at a time, parallel multiplications are avoided, thus resulting in a more hardware-efficient implementation. Fig. 2 shows a top-level schematic of the network, while Fig. 3, presents an indicative hidden layer with 2 neurons, including the intermediate shift-register used to connect consecutive layers.

A universal 5-bit counter informs the neurons about which sample is being processed at a given clock cycle, by controlling a multiplexer that supplies the appropriate neuron with the corresponding weight. The chosen weight and the incoming sample are multiplied to generate the product, pr .

$$pr_i = weight_i \times input_i \quad (1)$$

The accumulation of the products is performed by adding a feedback signal to the current product:

$$pr_acc_i = pr_i + feedback_i \quad (2)$$

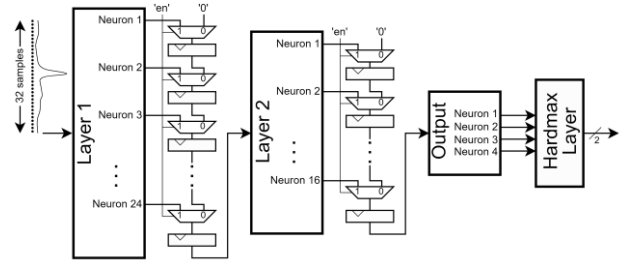


Fig. 2. Top-level schematic of the network.

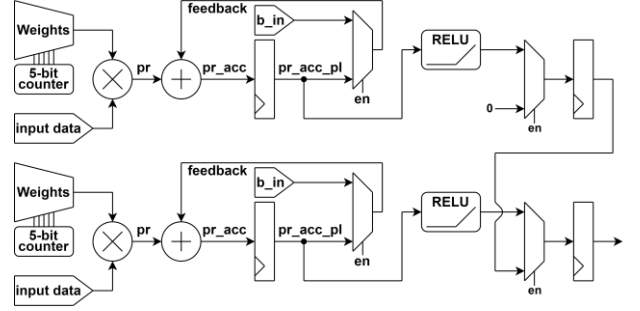


Fig. 3. Indicative hidden layer with two neurons.

The accumulated product, pr_acc_i , is stored in a register, and in the next clock cycle, appears as $pr_acc_pl_i$. This signal that represents the previous accumulated products, is added to the current product. There is however one exception. When the first sample is fed to the neuron, the counter is “0”, and triggers the signal, en . Thus, instead of adding $pr_acc_pl_i$, a multiplexer fetches the bias of the neuron. In other words, the multiplexer controls the value of the feedback signal:

$$feedback_i = \begin{cases} bias & \text{if } en = 1 \\ pr_acc_pl_i & \text{if } en = 0 \end{cases} \quad (3)$$

The accumulated products, that are stored in the register, are fed to the activation function, which in this case is ReLU. Implementing ReLU is a simple task, as it basically zeros a negative argument, while leaving unaffected a positive one:

$$ReLU_i = \begin{cases} 0 & \text{if } pr_acc_pl_i < 0 \\ pr_acc_pl_i & \text{if } pr_acc_pl_i \geq 0 \end{cases} \quad (4)$$

The enable signal drives one more multiplexer to determine the output of the neuron. When the system is ready to receive a new window, the counter is zeroed, the enable signal is *high*, and the final output of the previously processed window is allowed to pass through the multiplexer, to a Shift Register. The shift register receives the outputs of all the neurons in the layer, and on the next clock cycles, that enable is *low*, feeds them to the next layer, one sample at a time.

After the 4-neuron output layer, a Hardmax layer is used. Unlike the Softmax layer, it employs a hard modifier, as it sets *high* the vector with the largest value and *low* all the rest.

Each layer, including the Hardmax layer, processes its inputs in time frames of 32 clock cycles, equal to the number of neurons in the input layer. This pipelined architecture allows all layers to work in parallel. For instance, as shown in Fig. 4, while the first layer processes a given window w_0 , the second layer works on window w_{-1} until the current 32 clock-cycle frame ends, at which point it will move on to window w_0 . The second and third layers receive only 24, and 16 samples respectively, while the Hardmax layer requires only the final two clock cycles to produce an output. As a result, the interconnecting shift registers and the Hardmax layer remain idle for the time shown by the red rectangles in Fig. 4.

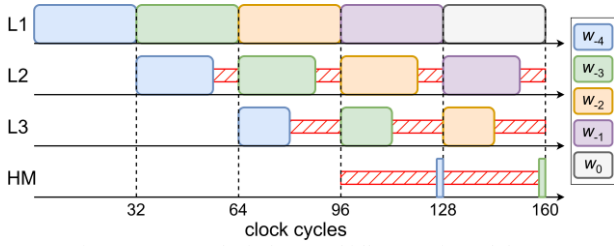


Fig. 4. Processed windows and idle states in each layer.

B. Binary Signals Fixed-Width Implementation

The representation of the binary vectors is a critical issue, as it has a direct impact on the accuracy drop with respect to the accuracy obtained by the original model (96.17%). Choosing a short bit-width for the fractional part of the signals increases the quantization error, as the floating-point parameters are heavily truncated. On the other hand, opting for longer signals results in heavy hardware requirements.

It has been shown that after the regularization, the integer part of all the obtained parameters can be described by three bits, without the risk of overflow. However, after exhaustive simulations, intermediate signals exceeding the range offered by three bits in the integer part, $[-4, 4)$, have been observed. Apart from aggressively adding one more bit at the left part of the radix, to tackle the occasional overflow problem, a more conservative solution has also been investigated. An overflow-detection and saturation module has been employed to single out the vectors that exceed the two limits. Once detected, these vectors are rounded to -4 and 4 (with the highest available precision) respectively.

Three different binary representations are considered in this work. The first one uses overflow-detection and saturation when needed, as it has 3 bits before the radix point and 5 after. These designs are referred to as 3.5. The second and third versions have 4 bits for the integer part to avoid overflowing signals, while for the fractional part, they use 6 and 8 bits and correspond to the representations 4.6 and 4.8, respectively. Clearly, the longer the bit-width, the higher is the achievable accuracy, as the saturation and quantization errors are completely or partially overcome.

C. Approximate Computing and Performances

Multiplying circuits consume a considerable amount of power and chip area. Each neuron in the proposed design employs a single multiplier, adding up to a total amount of 44 multipliers, that perform $(32 \cdot 24) + (24 \cdot 16) + (16 \cdot 4) = 1216$ multiplying operations per input. Approximate computing provides a good compromise between precision and hardware resources for error resilient and mobile applications. Aiming at reducing the hardware requirements, the methodologies from [21]–[23] have been applied to the multipliers of the NN. The best configurations are presented in Table III and Fig. 5.

Designs 3.5_Ex, 4.6_Ex, and 4.8_Ex use exact multipliers. Designs 3.5_4, 4.6_4, 4.8_4, and 4.8_5 use the approximate signed multipliers proposed in [21] with a segment of 6, 6, 8, and 10 bits respectively. The remaining designs employ 4×4 multiplying building blocks proposed in [22], to recursively generate the 8×8 , 10×10 , and 12×12 required multipliers. For the 10×10 multipliers in the 4.6 designs, $4:2$ approximate compressors proposed in [23] are also used.

As shown in Fig. 5 and Table III, using more bits for signal representation increases accuracy, by eliminating the

TABLE III: Accuracies and electrical performances.

Multiplier Design		Accuracy [%]	Power [μ W]	Area [μm^2]
8bits	3.5_1	64.68	0.107	2286
	3.5_2	68.72	0.108	2004
	3.5_3	69.06	0.113	2370
	3.5_4	76.52	0.129	2473
	3.5_Ex	76.62	0.211	2732
10bits	4.6_1	75.31	0.149	2613
	4.6_2	77.42	0.153	2667
	4.6_3	80.91	0.161	3115
	4.6_4	87.3	0.216	3121
	4.6_Ex	88.15	0.398	3928
12bits	4.8_1	86.16	0.271	3561
	4.8_2	87.64	0.276	3583
	4.8_3	90.24	0.278	3938
	4.8_4	93.25	0.434	4787
	4.8_5	93.35	0.548	5358
	4.8_Ex	93.35	0.731	5829

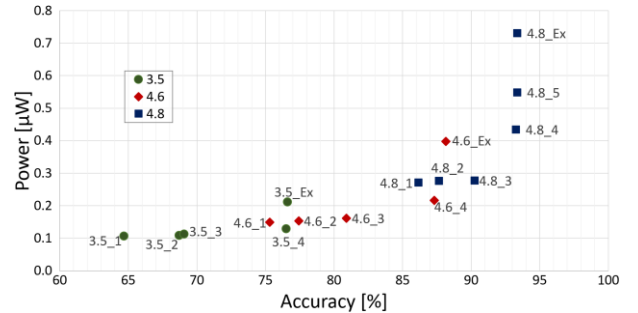


Fig. 5. Accuracy and power dissipation for different architectures.

overflow/saturation error and reducing the quantization error. Of course, power and area requirements increase as well.

All circuits are synthesized for a $10 \mu\text{s}$ timing constraint corresponding to 100 kHz. Power dissipation is obtained simulating the circuit with the dataset of 52000 windows to determine the switching activity. The power is reported for a single channel working at a frequency of 30 kHz that is the frequency needed for the given application that receives a new window every 6 input samples with a sample rate of 5 kHz. Hence, all windows will be centered enough for a correct elaboration. In fact, multiple readings of the same spike are possible and left to be filtered out by higher software layers, thus ensuring more robust behavior. The use of approximate computing results in significant reductions in area and power while not completely sacrificing accuracy. As an example, design 4.8_4 achieves 93.25% accuracy and reduces area by 18% w.r.t. 4.8_Ex that offers only 0.1% more accuracy.

When compared with the state-of-the-art (Table I), the proposed circuits compare favorably in terms of power and area (including the registers for the coefficients). When compared with the only integrated solution that provides both detection and classification, [11], the proposed circuit dissipates approximately 35 times less power while occupying a 40-times smaller silicon area.

IV. CONCLUSIONS

In this paper a dense neural network able to perform real-time spike detection and sorting is proposed and implemented. Training and validation data have been provided by NeuroCube, [24]. The proposed design is described in Verilog and synthesized for a 14nm technology. Approximate multipliers mitigate the hardware burden and a variety of circuits offering different tradeoffs have been presented.

REFERENCES

- [1] J.K. Chapin, K.A. Moxon, R.S. Markowitz, and M.A.L. Nicolelis, "Real-time control of a robot arm using simultaneously recorded neurons in the motor cortex," in *Nature Neuroscience*, vol. 2, no. 7, pp. 664 - 670, July 1999, doi: 10.1038/10223.
- [2] M.A. Lebedev, and M.A. Nicolelis, "Brain-machine interfaces: past, present and future," in *Trends in neurosciences*, vol. 29, no. 9, pp. 536 - 546, July 2006, doi: 10.1016/j.tins.2006.07.004.
- [3] A. B. Rapeaux, and T. G. Constandinou, "Implantable brain machine interfaces: first-in-human studies, technology challenges and trends," in *Current Opinion in Biotechnology*, vol. 72, pp. 102-111, Dec. 2021.
- [4] S. Saha et al., "Progress in brain computer interface: challenges and potentials," in *Frontiers in Systems Neuroscience*, vol. 15, p. 4, 2021.
- [5] J.A. Frank, M.J. Antonini, and P. Anikeeva, "Next-generation interfaces for studying neural function," in *Nature Biotechnology*, vol. 37, pp. 1013-1023, Sept. 2019, doi: 10.1038/s41587-019-0198-8.
- [6] M.A. Shaeri and A. M. Sodagar, "A Method for Compression of Intracortically-Recorded Neural...," in *IEEE Trans. on Neural Systems and Rehabilitation Engineering*, vol. 23, no. 3, pp. 485-497, May 2015.
- [7] G. Saggese, E. Zacharelos and A. G. M. Strollo, "Low Power Spike Detector for Brain-Silicon Interface using Differential Amplitude Slope Operator," in 17th PRIME Conference, June 2022, pp. 301-304.
- [8] G. Saggese, A.G.M. Strollo, "Low-Power Energy-Based Spike Detector ASIC for Implantable Multichannel BMIs," in *Electronics*, vol. 11, Sept. 2022, doi: <https://doi.org/10.3390/electronics11182943>.
- [9] G. Saggese et al., "Comparison of Sneo-Based Neural Spike Detection Algorithms for Implantable Multi-Transistor Array Biosensors," in *Electronics*, vol. 10, pp. 410, 2021, doi: 10.3390/electronics10040410.
- [10] T. Zhang, M. R. Azghadi, C. Lammie, A. Amirsoleimani, R. Genov, "Spike sorting algorithms and their efficient hardware implementation: a comprehensive survey," in *Journal of Neural Engineering*, vol. 20, Apr 2023, doi: 10.1088/1741-2552/acc7cc. PMID: 36972585.
- [11] D. Y. Barsakcioglu et al., "An Analogue Front-End Model for Developing Neural Spike Sorting Systems," in *IEEE Trans. on Biomedical Circuits and Systems*, vol. 8, no. 2, April 2014.
- [12] M. Rizk et al, "A fully implantable 96-channel neural data acquisition system," in *Journal of Neural Engineering*, vol. 6, May 2009.
- [13] J. Navajas et al, "Minimum requirements for accurate and efficient real-time on-chip spike sorting," in *Journal of Neuroscience Methods*, vol. 230, Apr. 2014, doi: <https://doi.org/10.1016/j.jneumeth.2014.04.018>.
- [14] J. Niediek, J. Bostrom, C.E. Elger, and F. Mormann, "Reliable analysis of single-unit recordings from the human brain under noisy conditions: tracking neurons over hours," in *PloS one*, vol. 11, no. 12, Dec. 2016.
- [15] M. S. Rehman et al., "SpikeDeeptector: A deep-learning based method for detection of neural spiking activity," in *Journal of Neural Engineering*, vol 16, July 2019, doi: 10.1088/1741-2552/ab1e63.
- [16] M. Tambaro et al., "A scalable spike detection method for implantable high-density multielectrode array," *PRIME 2021*, pp. 1-4, July 2021.
- [17] Z. Frehlick, I. Williams and T. G. Constandinou, "Improving neural spike sorting performance using template enhancement," in *IEEE Biomedical Circuits and Systems Conference (BioCAS)*, Shanghai, China, pp. 524-527, Oct. 2016, doi: 10.1109/BioCAS.2016.7833847.
- [18] S. L. Cam, H. Tran, R. Ranta and V. Louis-Dorr, "A Bayesian approach for Simultaneous Spike Extraction and Sorting," in 2019 9th International IEEE/EMBS Conference on Neural Engineering (NER), San Francisco, CA, USA, 2019, doi: 10.1109/NER.2019.8716888.
- [19] M. R  cz et al., "Spike detection and sorting with deep learning," in *Journal of Neural Engineering*, vol 17, no. 1, Jan. 2020.
- [20] M. S. Rehman et al., "SpikeDeep-Classifer: A deep-learning based fully automatic offline spike sorting algorithm," in *Journal of Neural Engineering*, vol 18, Febr. 2021, doi: 10.1088/1741-2552/abc8d4.
- [21] A.G.M. Strollo et al., "Approximate Multipliers Using Static Segmentation: Error Analysis and Improvements," in *IEEE Trans. on Circuits and Systems I: Regular Papers*, vol. 69, no. 6, pp. 2449-2462, June 2022, doi: 10.1109/TCSI.2022.3152921.
- [22] E. Zacharelos, I. Nunziata, G. Saggese, A. G. M. Strollo and E. Napoli, "Approximate Recursive Multipliers Using Low Power Building Blocks," in *IEEE Trans. on Emerging Topics in Computing*, vol. 10, no. 3, 1 July-Sept. 2022, doi: 10.1109/TETC.2022.3186240.
- [23] D. Esposito, A. G. M. Strollo, E. Napoli, D. De Caro and N. Petra, "Approximate Multipliers Based on New Approximate Compressors," in *IEEE Trans. on Circuits and Systems I: Regular Papers*, vol. 65, no. 12, pp. 4169-4182, Dec. 2018, doi: 10.1109/TCSI.2018.2839266.
- [24] M.L.A. Camu  as, and R.Q. Quiroga, "A detailed and fast model of extracellular recordings," in *Neural Computation*, vol 25, no. 5, pp. 1191-1212, May 2013, doi: https://doi.org/10.1162/NECO_a_00433.
- [25] E. Zacharelos, C. Scognamiglio, E. Napoli, and D. Gragnaniello https://github.com/dgragnaniello/on-chip_spike_detection